Module 2 : Maintenance Applicative

La maintenance applicative?

- Analyse => Compréhension
- Garantir son fonctionnement
- Ajouter des améliorations (sécurité, performance, code)
- Durée : vie de l'application

Risques

- Failles de sécurité
- Perte de performance
- Perte de temps

Cours 5 : Analyse, maintenance et améliorations 🔬

Consigne générale

- Noter les informations
- Expliquer les choix

Analyse 🔬

Pourquoi analyser?

- Comprendre le but
- Comprendre l'environnement
- Connaître les technologies utilisées

Analyse - Points clés

- Langage(s)
- Contexte et contraintes
- Profil des utilisateurs
- Prévoir la charge / rester cohérent

Recueillir des infos techniques

- Identifier langages (extensions, lecture rapide)
- Frameworks et gestionnaires de dépendances (composer.json, package.json)
- Fichiers/config spécifiques (ex. symfony.yaml)
- Dépendances externes (DB, serveurs, services) + versions
- Point d'entrée et autoload (ex. index.php)

Recueillir des informations fonctionnelles

- But général de l'application
- Fonctionnalités principales
- Acteurs et actions (ex. rôles, parcours)
- Méthodes: lire le code, naviguer dans l'app

Reproduction de l'application

Pourquoi reproduire l'appli ?

- Avoir un environnement de test
- Pouvoir reproduire les bugs
- Permet de trouver d'autres contraintes
- N'impacte pas la production

Maintenance et amélioration

Maintenance et amélioration

- Garantir le fonctionnement / sécurité
- Pas un ajout de nouvelles fonctions

Que maintenir?

- Versions langages & frameworks
- Le code
- Les fonctionnalités techniques

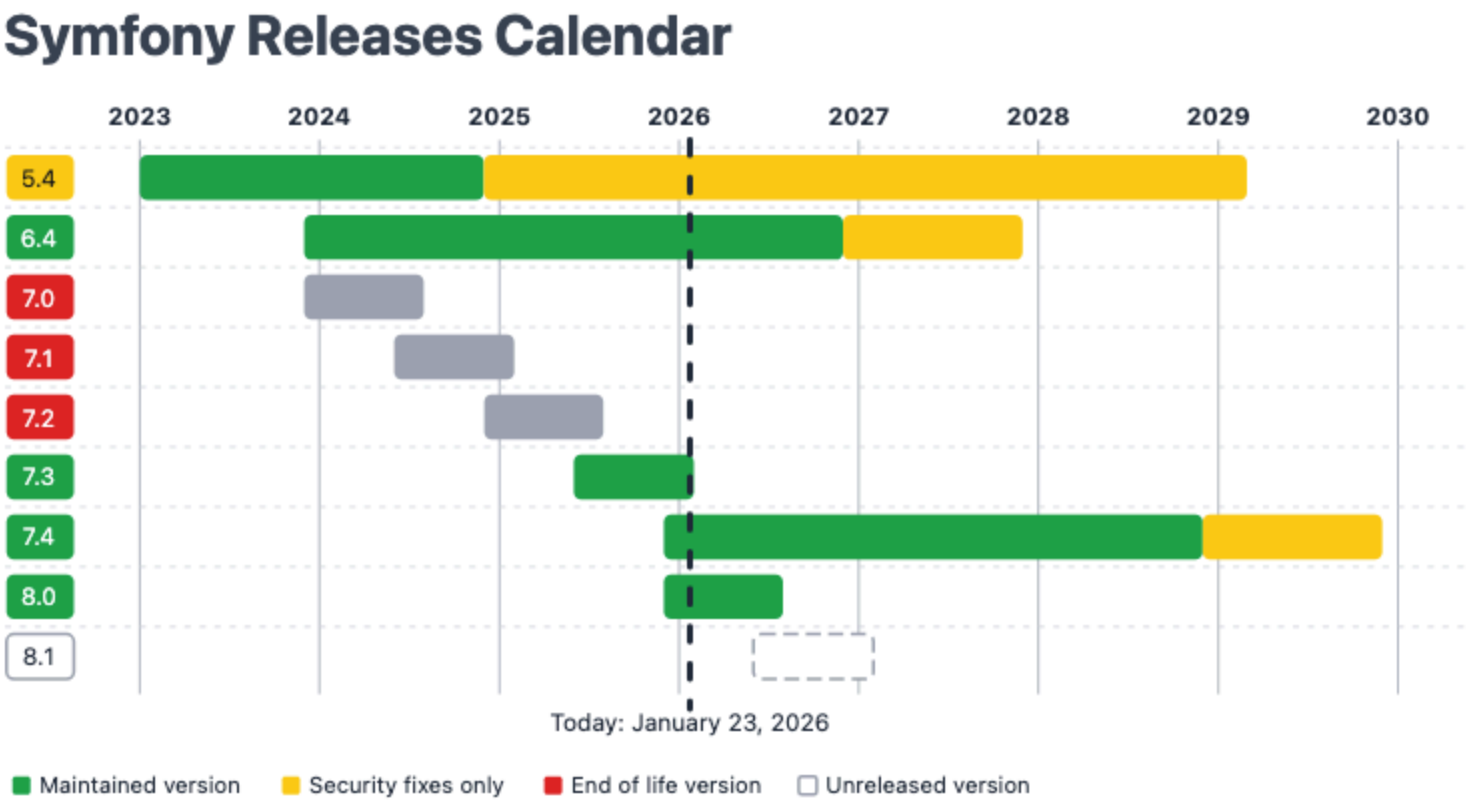
Langages et Frameworks

- Vérifier versions et compatibilité
- S'aider des gestionnaires de dépendances
- Monter de version de façon itérative

Version obsolète: principes

- Vérifier l'état de maintenance (langage, framework)
- Lire attentivement les release notes
- Préférer les montées mineures incrémentales
- Adapter composer.json / composer update au besoin

Ex: calendrier symfony



Solidification du code

- Vérifier les erreurs + Optimiser les performances
- Ex :
  - Typage et noms explicites
  - Fonctions dédiées pour cas précis
  - Gestion des exceptions (try/catch)
  - Nettoyage des inputs utilisateurs (sécurité)

Erreurs courantes dans le code

- Mots de passe en clair
- Droits ouverts / fichiers de configuration accessibles
- Variables non utilisées, noms incompréhensibles, etc.
- Frameworks ou logs dans le dépôt Git

Le code - attention

- Ne pas tout changer d’un coup
- Tester les changements et effets de bord
- Noter les modifications
- Isoler les changements dans des commits/branches spécifiques

Tests du code

- Important de tester le code pour éviter les régressions
- Tests unitaires
- Parfois complexe
- Modifier uniquement le code en cours d’évolution
- Utiliser les **baseline** si possible

Utilisation des logs

- Détecter erreurs, warnings, notices
- Repérer les fonctions dépréciées
- Corriger préventivement pour faciliter les upgrades

Configuration

- Éviter les valeurs codées en dur (MDP, URLs, mails)
- Déplacer dans fichiers de config / variables d'env
- Gérer multi-environnements sans if/else dans le code

Méthodes de travail

Bonnes pratiques

- Noter contexte, but, infos techniques
- Lister les idées avant d'implémenter
- Rationaliser et segmenter les changements
- Ne pas juger les anciens devs

Gérer le scope

- Segmenter le travail
- Todo lists pour suivre et revenir
- Éviter trop de chantiers simultanés

Résumé 📌

Étapes clés

- **Analyser**: technique (langages, frameworks, deps) + fonctionnel (but, fonctionnalités, acteurs)
- **Reproduire**: environnement maîtrisé (Docker Compose), point(s) d’entrée et config
- **Maintenir**: versions, logs, configuration externalisée
- **Solidifier**: typage, exceptions, sécurité des inputs
- **Documenter**: notes claires, partageables, pour revenir sereinement plus tard

À votre tour 💪